Question 19

Q: A self-driving car system relies on visual input. Explain how image acquisition and Computer Vision features contribute to reliable decision making.

Theoretical Concept: Vision in Self-Driving Cars

Acquisition: Cars use multiple high-fps, high-dynamic-range (HDR) cameras to handle extreme lighting transitions (like exiting a tunnel).

CV Features: Real-time Semantic Segmentation maps every pixel to a category (road, pedestrian). Stereo vision (multiple cameras) calculates depth. Reliable acquisition is paramount—if the camera is blinded by glare, algorithms fail.

💻 OpenCV Python Solution

Copy

```
import cv2  # Import OpenCV for image processing algorithms
import numpy as np                                          # Import Numpy for mathematical matrix operations
import matplotlib.pyplot as plt                             # Import Matplotlib to visualize the output on screen

# Step 1: Simulate left and right HDR camera frames from a car
imgL = np.zeros((100, 100), dtype=np.uint8)  # Create a blank black image array filled with zeros
imgR = np.zeros((100, 100), dtype=np.uint8)  # Create a blank black image array filled with zeros
```

```
# An obstacle appears slightly shifted due to parallax (Stereo Vision physics)
cv2.rectangle(imgL, (40, 30), (60, 70), 255, -1) # Object at x=40
cv2.rectangle(imgR, (30, 30), (50, 70), 255, -1) # Object at x=30

# Step 2: Initialize the Stereo Block Matching (StereoBM) algorithm
stereo = cv2.StereoBM_create(numDisparities=16, blockSize=15)  # Create stereo block matching object for depth esti

# Step 3: Compute the disparity (depth map) between the two cameras
disparity = stereo.compute(imgL, imgR)  # Compute disparity map from left and right stereo images

# Step 4: Display the depth map that the car's computer 'sees'
plt.imshow(disparity, cmap='plasma')  # Render the image array to the display
plt.title('Stereo Vision: Autonomous Depth Calculation')  # Add a descriptive title to the plot
plt.colorbar()  # Add a color legend bar to the plot
plt.show()  # Show the final plot window to the user
```

🎯 Expected Output

A bright yellow rectangle will appear on a purple background (using the 'plasma' colormap). The color mathematically represents the depth/proximity of the obstacle calculated from the dual camera images.

🚀 How to Execute & Submit

1. Google Colab Execution:

- Open Google Colab and create a New Notebook.

- Click the 📋 **Copy** button on the code block above.
- Paste it into a Colab cell and press **Shift + Enter** to run.

2. Uploading to GitHub:

- In Colab, go to **File** > **Save a copy in GitHub**.
- Authorize your GitHub account.
- Select your Computer Vision Lab repository and click **OK** to commit the code.